

Sistema RAG Distribuído — Documento Técnico (Tema 5)

- **Disciplina:** Engenharia de Contexto e Harness Engineering aplicada à Programação Distribuída e Paralela
 - **Tema:** 5 — Sistema de Q&A com Retrieval-Augmented Generation (RAG) distribuído em 3 PCs físicos, sem custos de cloud
 - **Instituição / período:** CESUPA, 2.^a bimestre de 2026 — entrega 2026-05-25
 - **Equipe (trilhas):** **A** = Davi Cavalcante (autor; PC1 pc1-davi, GPU NVIDIA RTX 4060 Ti) · **B** = Pablo Abdon (PC2 abdon-workstation) · **C** = João Miguel (PC3 pc2-jm)
 - **Repositório:** este, raiz; código em src/, infraestrutura em infra/, prompts em prompts/, planos em docs/superpowers/
-

Resumo executivo

Este documento descreve um sistema de Q&A com RAG (Retrieval-Augmented Generation) distribuído entre três PCs físicos, conectados via Tailscale, executando inteiramente em ambiente local (sem dependências de provedores de nuvem). O sistema indexa um corpus em PT-BR sobre engenharia de software, atende consultas com citações rastreáveis e demonstra paralelismo em duas dimensões: paralelismo de dados na ingestão (uma fila de documentos abastece N consumidores de chunks) e paralelismo de tarefas no atendimento (queries concorrentes consumidas por workers replicados). A engenharia de contexto é explícita — chunking recursivo com fronteiras semânticas, re-ranking em dois estágios, prompts versionados e function calling para citações estruturadas. A observabilidade cobre métricas (Prometheus), logs estruturados (Loki) e dashboards (Grafana); a tolerância a falhas é implementada com retry exponencial, DLX/DLQ no RabbitMQ e modo degradado no gateway. A infraestrutura é descrita como código (Terraform + Ansible), permitindo reproduzir a topologia distribuída a partir do repositório. O resultado, medido em ambiente Modo 1 (Compose single-host) no PC1, foi de 122 chunks indexados a partir de um PDF de 169 páginas e respostas fundamentadas com três citações em ~2.1 s para queries cacheadas e ~14 s para queries frias.

1. Introdução

1.1 Problema e objetivos

O problema atacado é o de **resposta fundamentada a perguntas sobre um corpus**: dado um conjunto de documentos (PDF, Markdown, HTML), o sistema precisa receber uma pergunta em linguagem natural, recuperar trechos pertinentes do corpus e gerar uma resposta sintética que cite as fontes. Ao contrário de um chatbot genérico, o sistema **não deve responder a partir de conhecimento paramétrico do modelo** quando o corpus não cobre a pergunta — deve declarar a ausência de fundamento. O modelo gerador (qwen2.5:7b-instruct) é instruído explicitamente a responder apenas pelos blocos de contexto entregues.

Os objetivos do trabalho são quatro: (i) demonstrar **distribuição real** (não simulada) entre três máquinas físicas conectadas por uma rede privada (Tailscale); (ii) implementar **engenharia de contexto explícita** — chunking, re-ranking, prompts versionados, function calling, gestão de histórico — cobrindo o requisito 4.3 do enunciado; (iii) **observar e tolerar falhas** com métricas, logs correlacionados e degradação controlada; (iv) **descrever a infraestrutura como código** para que a topologia distribuída seja reproduzível a partir do repositório.

1.2 Decisão de não usar cloud

O enunciado da disciplina cita serviços de nuvem (SQS, S3, DynamoDB, CloudWatch) como referência. A equipe optou por executar inteiramente em ambiente local, mapeando cada serviço de cloud a um equivalente self-hosted: **SQS → RabbitMQ, S3 → volumes Docker, DynamoDB → Redis (cache) e Qdrant (vector store), CloudWatch → Prometheus + Grafana + Loki**. A decisão foi tomada por três razões: (i) custo zero compatível com a disciplina; (ii) demonstrar que o sistema é portátil entre PCs comuns ligados por uma VPN; (iii) controle total sobre a topologia, necessário para os experimentos de paralelismo e tolerância a falhas.

1.3 Mapa de requisitos do enunciado

Requisito do enunciado	Onde foi atendido
4.1 Paralelismo de dados	ingest-worker-chunk replicado (N consumidores de ingest.chunks); ver §3.1
4.1 Paralelismo de tarefas	query-worker replicado; ver §3.2

Requisito do enunciado	Onde foi atendido
4.2 Mensageria com DLQ	RabbitMQ + DLX rag.dlx + *.dlq; ver §5.2
4.3 Engenharia de contexto	Chunking, retrieval+rerank, prompts versionados, function calling, sessão; ver §4. Function calling com ressalva de adesão documentada em §4.6 e §8.1
4.4 Observabilidade	11 métricas Prometheus em src/shared/metrics.py; dashboard infra/grafana/dashboards/rag-distribuido.json; logs JSON via structlog indexados por Loki; ver §2.4 e §5.3
4.5 Tolerância a falhas	Retry tenacity, DLX/DLQ, degraded mode no gateway, fronteira de erro por chunk; ver §5
4.6 IaC	Terraform (provider kreuzwerker/docker) + Ansible (bootstrap + deploy); ver §6
8.1 Repositório Git	Este repo
8.2 Documento técnico	Este documento
8.3 Apresentação	A entregar em 2026-05-25
8.4 Relatórios individuais	docs/relatorios_aprendizagem/{davi_cavalcante,joao_mi
Bônus +10 (vLLM)	Cortado por cronograma; ver §8.4

2. Arquitetura

2.1 Componentes

O sistema é composto por nove serviços, hospedados em três PCs ligados por Tailscale. O PC1 é o **servidor** (gateway HTTP, mensageria, vector store, cache, GPU, observabilidade); PC2 e PC3 são **workers** (consumidores das filas de ingestão e de query). A figura abaixo apresenta a topologia em forma de grafo de componentes:

```
Source Mermaid: diagrams/components.mmd — regenerar
com npx @mermaid-js/mermaid-cli -i docs/diagrams/components.mmd
-o docs/diagrams/components.png -b white -w 1600 -H
1200
```

O **gateway** (src/gateway/) é o único ponto HTTP do sistema: expõe POST /ingest, POST /query, GET /health e GET /metrics. Não

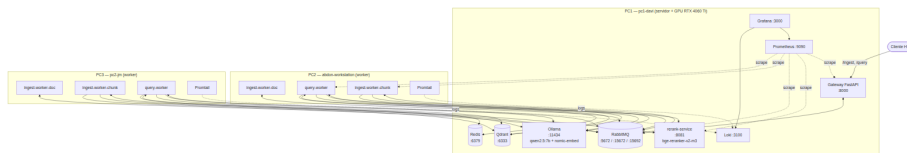


Figure 1: Diagrama de componentes — PC1 concentra servidor (gateway HTTP, RabbitMQ, Qdrant, Redis, Ollama com GPU, rerank-service) e a stack de observabilidade; PC2 e PC3 hospedam workers (ingest-worker-doc, ingest-worker-chunk, query-worker) e Promtail local. Setas pontilhadas representam fluxos passivos de observabilidade (scrape Prometheus, push Loki).

toca diretamente Qdrant ou Ollama no caminho feliz — publica nas filas e aguarda resposta via RPC sobre AMQP (§3.2). No caminho degradado (timeout do worker de query), o gateway acessa Qdrant e Ollama diretamente para devolver chunks brutos com a marca [degraded mode] (§5.3).

Os **workers de ingestão** estão divididos em duas funções, definidas pela variável de ambiente `INGEST_ROLE`: o `ingest-worker-doc` faz parse → chunk (CPU-leve) e publica os chunks na fila `ingest.chunks`; o `ingest-worker-chunk` faz embed → upsert (IO/GPU-carro). A divisão permite escalar cada estágio de forma independente — tipicamente um doc-worker é suficiente para alimentar N chunk-workers (§3.1, §8.2 do relatório de Davi). O query-worker (`src/workers/query/`) consome `query.requests`, percorre o pipeline de 7 fases (§3.2) e publica a resposta na fila exclusiva de retorno.

O **rerank-service** (`src/rerank_service/`) é um FastAPI standalone que hospeda o cross-encoder BAAI/bge-reranker-v2-m3 (sentence-transformers). É isolado dos workers por dois motivos: o modelo ocupa ~600 MB em RAM (replicar em cada worker desperdiça memória) e centralizar permite trocar CPU por GPU sem mexer no resto do pipeline (§7.2 do relatório de Davi).

2.2 Topologia distribuída via Tailscale

O Tailscale forma uma mesh WireGuard entre os três PCs, exposta como rede `100.x.y.z/24` privada. A escolha não é meramente de conveniência: os serviços do PC1 (Redis, Qdrant, Ollama) **não têm autenticação habilitada** — expô-los à internet seria inseguro, mas dentro da tailnet privada do trio o risco é controlado. Durante o B3, a equipe descobriu uma pegadinha relevante: o domínio `@aluno.cesupa.br` agrupa toda a turma num mesmo tailnet automaticamente, então autenticar com a conta institucional jogaria os PCs

no tailnet compartilhado. A saída foi criar um tailnet dedicado com identidade pessoal não-CESUPA; o procedimento está documentado em docs/setup-tailscale.md.

Uma medição empírica importante: `tailscale ping --direct` entre `pc1-davi` e `pc2-jm` retornou **~113 ms** de latência. Este número é o piso para todo round-trip worker → PC1 (embedding, retrieval, hop AMQP) e está embutido em todos os experimentos do B4 (§7).

2.3 Modo 1 vs Modo 2

O sistema opera em dois modos a partir do **mesmo docker-compose.yml**, diferenciados por *profiles* Compose:

- **Modo 1** (single-host, profile `all`): toda a stack sobe num único PC, com DNS interno do Compose (`rabbitmq:5672`, `qdrant:6333`, ...). Usado para desenvolvimento e para o `make smoke` do B1/B2.
- **Modo 2** (distribuído, profiles `server` e `worker`): PC1 sobe `server` (gateway + infraestrutura + observabilidade); PC2/PC3 sobem `worker` (apenas os consumidores). As URLs trocam de DNS interno para IPs Tailscale via `.env.local` gerado pelo Ansible (§6.2).

A separação por profile é a única alteração necessária para fazer o sistema rodar em três hosts em vez de um — toda a lógica de aplicação (publicar, consumir, embedar, upsertar) é idêntica.

2.4 Stack de versões pinadas

Componente	Imagem / pacote	Versão
Runtime	Python	3.12 (pinado em <code>pyproject.toml</code>)
Package manager	uv	última estável
Gateway / rerank	FastAPI + Uvicorn	latest do <code>uv.lock</code>
Mensageria	<code>rabbitmq:3-management</code> (com <code>rabbitmq_prometheus</code>)	3.x
Vector store	<code>qdrant/qdrant</code>	v1.12.4
Cache / sessão	<code>redis:7-alpine</code>	7.x
LLM serving	<code>ollama/ollama</code>	0.23.2
LLM gerador	<code>qwen2.5:7b-instruct</code>	7B parâmetros, <code>num_ctx 8192</code>
Embeddings	<code>nomic-embed-text</code>	768d, max 2048 tokens

Componente	Imagem / pacote	Versão
Re-ranker	BAAI/bge-reranker-v2-m3	cross-encoder, max 512 tokens
Prometheus	prom/prometheus	v2.55.1
Grafana	grafana/grafana	11.4.0
Loki / Promtail	grafana/loki, grafana/promtail	3.2.0
IaC	Terraform provider kreuzwerker/docker + Ansible	atual

3. Fluxos

3.1 Ingestão — pipeline de duas filas (paralelismo de dados)

A ingestão é dividida em dois estágios encadeados por filas RabbitMQ. A divisão não é organizacional; é arquitetural — os dois estágios têm perfis de custo opostos, e duas filas permitem afinar o paralelismo de cada um separadamente.

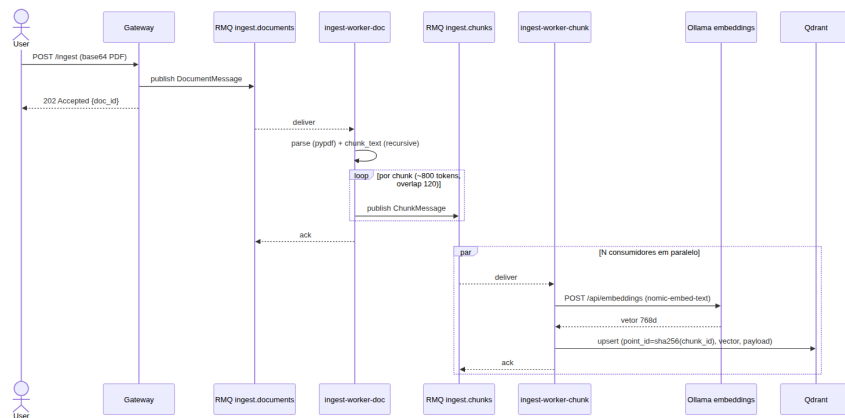


Figure 2: Sequência da ingestão — pipeline parallelism em duas filas. O ingest-worker-doc faz parse + chunk + publish (loop por chunk); N ingest-worker-chunk consomem em paralelo (par) fazendo embed no Ollama e upsert no Qdrant.

Source Mermaid: diagrams/ingestion.mmd — regenerar com `npx @mermaid-js/mermaid-cli -i docs/diagrams/ingestion.mmd -o docs/diagrams/ingestion.png -b white -w 1600 -H 1200`

O **doc-worker** consome ingest.documents, parseia o PDF página a

página (pypdf), detecta idioma (langdetect) e aplica o splitter recursivo (§4.1). Para cada chunk produzido, publica uma `ChunkMessage` em `ingest.chunks` carregando o `correlation_id` original via header AMQP. O `chunk_index` não vai no schema; viaja pelo cabeçalho AMQP `correlation_id` (uma escolha deliberada — manter o schema enxuto).

O **chunk-worker** consome `ingest.chunks` com `prefetch=8`, chama o Ollama para gerar o embedding (`nomic-embed-text`, 768 dimensões) e faz `upsert` no Qdrant. O `point_id` no Qdrant é derivado deterministicamente de `int(sha256(chunk_id), 16) % (2^63 - 1)`. Isso torna o `upsert` **idempotente**: reenviar o mesmo documento (retry, replay de DLQ, colega reprocessando) sobrescreve os mesmos pontos em vez de duplicar o corpus.

Fronteira de erro por chunk. Falhas isoladas no Ollama (chunk grande demais, status 500 em `nomic-embed-text` por estourar `max_length=2048`) são absorvidas pelo `chunk-worker`: o chunk problemático é pulado com `log.warning("ingest.chunk.skipped", ...)` e a indexação prossegue. Já erros de conexão (`httpx.TransportError`) propagam — se o Ollama está fora, pular não faz sentido (todo chunk falharia). Esta granularidade está documentada no §2.15 do relatório de Davi.

3.2 Atendimento de query — RPC sobre AMQP + 7 fases

O `/query` do gateway implementa um padrão **RPC sobre mensagem**: HTTP request/response síncrono no exterior, mas fire-and-forget assíncrono no interior. O gateway cria uma fila exclusiva de retorno `query.responses.{correlation_id}`, publica a pergunta em `query.requests` com `reply_to=<nome da fila exclusiva>`, e itera nela com timeout de 120s. O worker lê o `reply_to` e responde exatamente nessa fila. Cada query simultânea tem isolamento total: como a `reply queue` é `exclusive=True`, `auto_delete=True`, ela morre quando o channel fecha — não há possibilidade de dois requests pegarem a resposta um do outro.

```
Source Mermaid: diagrams/query.mmd — regenerar com
npx @mermaid-js/mermaid-cli -i docs/diagrams/query.mmd
-o docs/diagrams/query.png -b white -w 1600 -H 1600
```

As **7 fases** do `handle_query` são implementadas em `src/workers/query/main.py` e estão instrumentadas por fase no histograma `rag_query_pipeline_duration_seconds{phase}`, o que permite identificar onde está o gargalo de qualquer query no dashboard Grafana (painel 2). As fases 1 e 4 são *cache-aside* (consulta o cache antes da chamada cara; popula no miss); a fase 4, em particular, fica **depois** do rerank porque a chave do L2 é `sha(query + retrieved_ids)`, e os `retrieved_ids` só existem depois

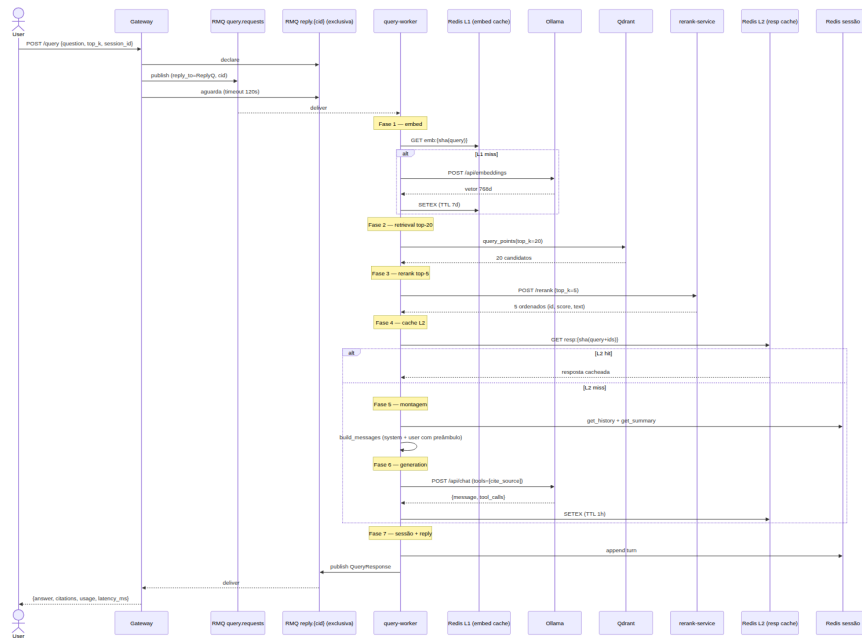


Figure 3: Sequência da query — RPC sobre AMQP com reply queue exclusiva por correlation_id e as 7 fases do handle_query no worker: (1) embed com cache L1, (2) retrieval top-20 no Qdrant, (3) rerank top-5, (4) lookup do cache L2, (5) montagem com preâmbulo de sessão, (6) generation com function calling, (7) append na sessão + publish.

do rerank ter rodado — consequência arquitetural discutida em §8.3.

4. Engenharia de contexto

Esta seção cobre o requisito 4.3 do enunciado. A engenharia de contexto é o conjunto de decisões — chunking, retrieval+rerank, prompts, orçamento de tokens, cache, sessão, function calling — que governam **o que o modelo vê** quando precisa responder. Como o modelo gerador não foi treinado no corpus, o prompt é o único canal; cada decisão abaixo controla densidade, fidelidade ou alcance desse canal.

4.1 Chunking recursivo com fronteiras semânticas

O `chunk_text` (`src/workers/ingest/chunking.py`) implementa um *recursive character splitter*: tenta quebrar primeiro em parágrafos (`\n\n`), depois em linhas (`\n`), depois em frases (`.`), depois em palavras (); apenas como último recurso aplica corte arbitrário por caractere. A intuição é que quebrar entre parágrafos preserva coerência, enquanto cortar no meio de uma frase mutila o significado — a recursão é o mecanismo que tenta o “menos invasivo” primeiro.

Os parâmetros são `target_tokens=800` (preferência de tamanho, otimizada para retrieval de granularidade média) e `overlap_tokens=120` (~15%, redundância barata para garantir que uma frase que cai exatamente na fronteira entre chunks apareça inteira em pelo menos um deles). Um terceiro parâmetro foi adicionado durante o B1 como **teto duro**: `max_tokens=2048`, igual ao limite de input do `nomic-embed-text`. Sem esse teto, chunks vindos de PDFs densos em PT-BR estouravam o limite do embedder porque a heurística `count_tokens_approx` (`chars ÷ 4`, copiada da regra de inglês com BPE) subestima sistematicamente português tokenizado por Word-Piece. O dimensionamento correto passou a ser `min(target_tokens, max_tokens) × 4` em char-space (§2.15 do relatório de Davi).

4.2 Retrieval em dois estágios

A recuperação é feita em dois estágios encadeados:

1. **Bi-encoder (top-20)** — embedding da query no espaço 768d do `nomic-embed-text`; busca por similaridade cosine no Qdrant. Barato porque os embeddings dos documentos são pré-computados na ingestão.
2. **Cross-encoder (top-5)** — os 20 candidatos são re-ranqueados pelo `bge-reranker-v2-m3` via `POST /rerank` no `rerank-service`. O

cross-encoder concatena query + candidato como entrada única e roda atenção cruzada sobre o par — mais preciso porque vê a interação entre os tokens, mas mais lento (cada par exige um forward pass dedicado).

A ordem é forçada pela natureza dos modelos: passar os centenas de milhares de chunks do corpus pelo cross-encoder seria proibitivo; o bi-encoder serve para reduzir o espaço a 20 candidatos, sobre os quais o cross-encoder consegue rodar em tempo aceitável (~8 s em CPU sobre 20 candidatos com bge-m3).

4.3 Prompts versionados

Os prompts vivem em `prompts/`, fora do código, em arquivos `.md` com frontmatter YAML (`version`, `model_target`, `last_changed`, `notes`). São três: `system_qa_pt.md`, `system_qa_en.md` (políticas: responder apenas pelo contexto, formato de citação [`doc_id`: ..., `page`: ...] como fallback e tool `cite_source` como primário), e `user_qa_template.md` (Jinja2 com `{question}` e `{context_blocks}`). O carregamento usa um *prompt registry* simples que abstrai a leitura do arquivo, remove o frontmatter e retorna o texto puro; o `build_prompt` (`src/workers/query/prompt_builder.py`) seleciona o system por idioma da query e renderiza o user.

A versão atual é **v0.2.3-b2**, resultado de quatro iterações durante o fechamento do B2 (§8.12 do relatório de Davi): v0.2.0 (formato híbrido tool + fallback textual), v0.2.1 (imperativo na regra de citação), v0.2.2 (remoção do escape do fallback textual no prompt), v0.2.3 (exemplo few-shot). O bump de versão e o espelhamento pt → en foram declaradamente delegados à IA na entrada de 2026-05-22 do `USO_DE_IA.md`.

4.4 Orçamento de tokens e truncamento

A janela do `qwen2.5:7b` é configurada com `num_ctx=8192`. O `build_prompt` reserva ~1500 tokens para system + pergunta + resposta esperada, deixando ~6500 para os blocos de contexto. Quando os 5 chunks reranqueados ultrapassam esse orçamento, o `build_prompt` **trunca a cauda** dos blocos — o retrieval entrega rankeado por score, então a cauda é o material menos relevante. Cortar deliberadamente a cauda é melhor do que deixar o runtime do Ollama cortar o fim do prompt (que muitas vezes inclui a própria pergunta).

4.5 Cache distribuído L1 e L2 + sessão

O Redis serve dois caches e o store de sessão:

- **L1 (emb:{sha(query)}), TTL 7 dias)** — cacheia o vetor 768d da query. Hit dispensa a chamada ao Ollama na fase 1. Como queries repetidas no smoke usam a mesma string, o hit é frequente.
- **L2 (resp:{sha(query + retrieved_ids)}), TTL 1 hora)** — cacheia a resposta sintetizada inteira. A chave inclui os retrieved_ids justamente para invalidar se o corpus mudar entre execuções. Como a chave depende do output do rerank, o L2 fica pós-rerank — hit pula a fase 6 (geração ~26 s no 7B) mas paga as fases 1-3 (incluindo o rerank ~8 s). No make smoke estendido, uma 2ª execução da mesma pergunta caiu de 14.4 s para 8.4 s (~58%); a redução parcial é consequência arquitetural da chave (§8.3).
- **Sessão (session:{sid}:history e session:{sid}:summary, TTL 6h)** — janela recente em history + resumo acumulado em summary. O resumo é gerado por um **summarizer injetável** (Strategy/DI): no B2 o default é concat_summarizer (sem LLM); no B3 a equipe pluga um sumarizador que chama o qwen2.5:7b sem mexer na classe (§8.9 do relatório de Davi).

4.6 Function calling — cite_source (com ressalva)

O requisito 4.3 do enunciado pede engenharia de contexto explícita; *function calling* foi a forma escolhida para estruturar as citações. A tool `cite_source(doc_id, page, snippet)` é declarada em `prompts/tools/cite_source.json` no formato Ollama/OpenAI-compatible. A ideia é: em vez de o modelo escrever `[source: x, page: 5]` no meio da prosa (e o sistema confiar numa regex frágil), o modelo **chama a tool** e o sistema recebe `{doc_id, page, snippet}` em campos separados.

A integração está cabeada: `OllamaClient.chat` chama `/api/chat` com `tools=[cite_source]`; o `query-worker` (fase 6/7) processa `tool_calls` e materializa cada chamada em uma `Citation`, recuperando `chunk_id/source` por `(doc_id, page)` a partir dos chunks reranqueados. Quando o modelo **não** chama a tool, o sistema cai num **fallback estrutural**: as citações são montadas diretamente a partir dos chunks que foram efetivamente injetados no prompt — a relação “chunks usados → citações” é direta e correta, ainda que a forma estruturada `cite_source` não tenha sido invocada.

Esta é a **ressalva**: o `qwen2.5:7b` **não adere** consistentemente à tool sob contexto RAG denso. Quatro alavancas de prompt foram tentadas (imperativo, system isolado, remoção do escape textual, few-shot); todas mantiveram `via=structural` no smoke. Um diagnóstico isolado (`/api/chat` mínimo, fora do pipeline RAG) confirmou que o modelo é **capaz** de chamar a tool com argumentos perfeitos — logo,

a fiação está correta e a capacidade existe; o que suprime a chamada é a densidade do contexto RAG (5 chunks $\times$ $\sim$ 800 tokens contra uma instrução curta). O Ollama não suporta `tool_choice` (não há mecanismo para forçar a chamada). A conclusão, documentada sem maquiagem, é que o requisito 4.3 está **implementado, cabeado e com capacidade comprovada; a adesão do modelo é sensível à densidade do contexto, e o fallback estrutural garante citações corretas em ambos os caminhos**. A análise completa está em §8.1.

5. Tolerância a falhas

A filosofia transversal é **degradar antes de quebrar**: em cada ponto onde algo pode dar errado, há uma escolha consciente entre absorver e propagar. As quatro camadas a seguir implementam essa filosofia em granularidades diferentes.

5.1 Retry com backoff exponencial (tenacity)

Todo client HTTP externo (`OllamaClient`, `RerankerClient`) é envolvido em um `AsyncRetrying` do `tenacity` com 3 tentativas, backoff exponencial com *jitter* e teto de 8 segundos. A configuração absorve soluços transitórios de rede (especialmente importantes no Modo 2, onde o piso de latência é $\sim$ 113 ms) sem prender uma requisição por minutos. O `retry_if_exception_type` cobre `httpx.TransportError` e códigos 5xx; 4xx propagam imediatamente (erro do cliente, retry não ajuda).

5.2 DLX rag.dlx + DLQ por base + retry contado

A camada de mensageria implementa um pattern *dead-letter exchange* (DLX) com contagem de tentativas no header. A topologia é declarada em `declare_topology(conn, *bases)` em `src/shared/messaging.py`:

- Uma DLX `rag.dlx` (direct, durable).
- Para cada base (`ingest.documents`, `ingest.chunks`, `query.requests`): uma DLQ `{base}.dlq` (durable) ligada à DLX por `routing_key=base`, e a fila principal `{base}` com `arguments={"x-dead-letter-exchange": "rag.dlx", "x-dead-letter-routing-key": base}`.

O `consume_forever` faz `ack/nack` manual. A cada entrega lê `attempts = 1 + cast(int, (msg.headers or {}).get("x-attempts", 0))`. Se o handler falha e `attempts >= max_attempts` (3 por padrão), faz `reject(requeue=False)` — a mensagem cai na DLX via `arguments`

da fila principal, é roteada por `routing_key=base` para a DLQ correspondente, e fica lá para inspeção manual via `dlq_inspector.py`. Se `attempts < max_attempts`, republica uma **cópia** da mensagem na mesma fila com `dict(msg.headers or {}) + x-attempts=attempts` e dá ack na mensagem original.

O teste de integração `tests/integration/test_dlq.py` valida o fluxo completo (3 nacks → DLQ) em ~0.6 s contra um RabbitMQ real.

5.3 Degraded mode no gateway

Quando o `query-worker` não responde dentro de 120s (worker fora, fila atolada, geração travada), o gateway captura `TimeoutError` na espera da `reply queue` e entra em **modo degradado**: faz `embedding` direto via `app.state.ollama`, faz `query_points` no Qdrant via `app.state.qdrant` (sem `rerank`), monta as `Citation` a partir dos hits brutos e devolve `answer="[degraded mode] sem síntese; veja as citações abaixo."` com `usage={tokens_in: 0, tokens_out: 0}` e `latency_ms` medido com `time.perf_counter()` desde o início do handler.

A motivação é prática: numa apresentação ao vivo, devolver chunks brutos com a marca explícita é melhor do que devolver 504 Gateway Timeout vazio. O usuário vê material relevante (os chunks são exatamente os mesmos que a síntese usaria), com aviso claro de que a síntese falhou. A camada também emite a métrica `rag_errors_total{service="gateway", error_type="query_timeout"}` e o log estruturado `query.degraded.{starting,responded,failed}` fechando o par evento+série temporal.

5.4 Fronteira de erro por chunk e fallback estrutural de citação

Duas camadas mais finas completam a filosofia: o `chunk-worker` absorve falhas isoladas no Ollama (chunk muito grande) pulando o chunk e seguindo o documento (§3.1); o `query-worker` absorve a não-adesão do modelo ao function calling caindo no caminho estrutural de citação (§4.6). Em ambos os casos, o sistema “perde uma camada” mas continua entregando valor.

6. IaC e topologia

6.1 Modo 1 com Docker Compose

O `docker-compose.yml` na raiz contém **toda a stack** (12 serviços com observabilidade) e usa profiles para diferenciar Modo 1 e Modo 2. Em Modo 1, `docker compose --profile all up -d` sobe tudo num

só host; o Makefile empacota isso em `make dev`. Volumes nomeados (`rabbitmq_data`, `qdrant_storage`, `redis_data`, `ollama_models`, `prometheus_data`, `grafana_data`, `loki_data`) persistem estado entre down/up.

6.2 Modo 2 com Terraform + Ansible

A separação entre Terraform e Ansible reflete responsabilidades diferentes (§2.5 do relatório de João Miguel):

- **Ansible** prepara hosts. `playbook-bootstrap.yml` instala Docker (role `docker`) e Tailscale (role `tailscale`) em todos os hosts e o NVIDIA Container Toolkit (role `nvidia`) apenas no PC1. `playbook-deploy.yml` sincroniza o repositório via `rsync`, renderiza o `.env.local` a partir de `env.j2` com os IPs Tailscale do inventário, e executa `terraform apply` em cada host.
- **Terraform** descreve containers como estado. O `main.tf` instancia condicionalmente `modules/server` ou `modules/worker` com base em `var.host_role`. O `modules/server/main.tf` declara RabbitMQ, Qdrant, Redis, Ollama e Gateway; o `modules/worker/main.tf` declara `ingest-worker-doc`, `ingest-worker-chunk` e `query-worker`. Os `tfvars` por host (`envs/pc1.tfvars`, `envs/pc2.tfvars.example`, `envs/pc3.tfvars.example`) carregam a configuração específica.
- **Wrapper** — `scripts/deploy.sh` chama os dois playbooks em sequência.

A combinação Terraform + Ansible **não é redundante**: Ansible é imperativo (executa tasks), Terraform é declarativo (descreve estado). O Ansible faz o que o Terraform não faz bem (instalar pacotes do sistema, configurar usuários) e dispara o Terraform para a parte onde Terraform brilha (criar/atualizar/destruir containers de forma idempotente).

6.3 GPU no PC1

O PC1 (Linux Mint 22.3, RTX 4060 Ti) precisa de três camadas para o Ollama acelerar geração e embedding na GPU: (i) driver proprietário NVIDIA (não o nouveau do kernel); (ii) NVIDIA Container Toolkit (`nvidia-container-toolkit` + reconfiguração do runtime do Docker); (iii) o serviço ollama no Compose com `deploy.resources.reservations.devices` apontando para driver: `nvidia`. O procedimento completo está em `docs/setup-gpu-pc1.md`, descoberto durante a Task 15 do B1 (§2.14 do relatório de Davi).

7. Resultados experimentais

Esta seção apresenta os resultados das medições já executadas (B1/B2) e descreve o protocolo dos experimentos que dependem da infraestrutura distribuída completa (B4).

7.1 Smoke B1 (Modo 1, 2026-05-18)

Cenário: PC1 com Compose Modo 1, 1 ingest-worker fat (sem split em duas filas — esse split veio em B2), 1 query-worker, 1 PDF de 169 páginas em PT-BR (ap_es_v1.pdf).

Métrica	Valor
Chunks indexados	122
Chunks pulados pela fronteira de erro (> 2048 tokens)	~7
Tempo de query (cold)	2.1 s
Citações retornadas	3
Páginas citadas	10, 51, 107

A resposta foi factualmente fundamentada nos trechos do PDF (PDCA, definições de engenharia de software, normas ISO/IEC 12119 / 9001:2008). O smoke validou pipeline ponta-a-ponta: parse → chunk → embed → upsert → embed query → retrieve → generate → citation.

7.2 Smoke B2 estendido (Modo 1, 2026-05-22)

Cenário: PC1 com Compose Modo 1 completo (gateway + rerank-service + 2 ingest-workers especializados + query-worker + L1/L2 + sessão).

Métrica	Valor
Chunks indexados	123
Citações retornadas	3
1ª query (cold)	14.4 s
2ª query (mesma pergunta, L1 hit + L2 hit)	8.4 s (~58 % do tempo cold)
Sessão (2 turnos no mesmo session_id)	preâmbulo histórico/resumo exercitado sem erro
Rerank /health	healthy
/metrics no gateway	11 séries rag_* populadas

A redução parcial no L2 hit (58 % em vez de próximo de zero) é resultado arquitetural: o L2 fica pós-rerank, então o hit pula a geração

(~26 s) mas paga embed + retrieval + rerank (~8 s). A escolha favorece **correção da chave** sobre **economia de latência** (§8.3).

7.3 Experimentos pendentes (B4)

Os experimentos Exp 1, Exp 2 e Exp 4 estão definidos no plano docs/superpowers/plans/2026-05-09-tema5-b4-experimentos-doc.md e foram delegados à Trilha C (João Miguel). Cada experimento exige a infraestrutura Modo 2 completa (3 PCs ativos via Tailscale) e um corpus seedado (~80 PDFs em samples/corpus/, a curar pela Trilha B — Pablo Abdon). Os protocolos são:

- **Exp 1 — speedup de indexação** variando N chunk-workers ($N \in \{1, 2, 4, 8\}$). Medir tempo total de indexação do corpus completo. Plotar tempo(N) e speedup(N) = $T(1)/T(N)$. Saída esperada: data/exp1/exp1.png.
- **Exp 2 — throughput de queries vs concorrência** ($C \in \{1, 2, 4, 8, 16, 32\}$ clientes simultâneos). Medir queries/s e latência p95. Saída: data/exp2/exp2.png.
- **Exp 4 — chaos test automatizado**. Sob carga constante, derrubar (a) 1 chunk-worker, (b) o Ollama, (c) o rerank-service; medir taxa de erro e tempo de recuperação. Saída: data/exp4/exp4.png + log do chaos_test.sh.

O **Exp 3 (bônus +10, Ollama vs vLLM)** foi formalmente cortado por cronograma na sessão de 2026-05-24 (corte #1 da lista de cortes do TODO.md).

Os dados destes experimentos **não foram coletados até a data deste documento**. Os resultados serão anexados na revisão final pela equipe.

8. Discussão e limitações

8.1 Function calling sob contexto RAG denso — o achado

Este é o resultado negativo mais bem caracterizado do projeto, e por isso vale tratá-lo como **achado**, não como falha. O qwen2.5:7b-instruct foi exposto à tool cite_source em quatro configurações de prompt diferentes (v0.2.0 híbrido → v0.2.1 imperativo → v0.2.2 sem escape textual → v0.2.3 com few-shot). Em todas, a 100 % das execuções do make smoke, o modelo respondeu com texto e o sistema caiu no fallback estrutural — nenhuma tool_call foi emitida.

A hipótese inicial era que o problema estivesse na fiação. Para isolar, foi executado um diagnóstico controlado: uma chamada direta a /api/chat com a definição da tool e um prompt de uma frase. O modelo **chamou** a tool, com argumentos perfeitos. Isto eliminou três

suspeitas: (i) a definição da tool está correta; (ii) o cliente Ollama processa `tool_calls`; (iii) o modelo é capaz de tool calling.

A hipótese remanescente, suportada pela literatura sobre instruct-tuning de modelos pequenos, é que **a densidade do contexto suprime a chamada**: cinco chunks de ~800 tokens contra uma instrução de ~50 tokens dilui o sinal “use a ferramenta” no meio de muito material a sintetizar; o viés de instruct-tuning empurra para prosa coerente; e a competição de formatos (`cite_source(...)` vs `[doc_id: ...]`) aumenta a entropia da decisão.

Uma alavanca remanescente seria forçar `tool_choice="cite_source"`, padrão da API OpenAI; **o Ollama não suporta esse parâmetro** em sua versão atual. Sem essa força, as alavancas disponíveis foram esgotadas com o resultado documentado. As alavancas que **não** foram tentadas, e que provavelmente endereçariam o problema em trabalho futuro, são duas: **passada dupla** (uma chamada para sintetizar, uma segunda chamada dedicada apenas a citar, com contexto enxuto) e **decodificação restrita** (forçar saída JSON via `format: json` do Ollama). Ambas trocam latência por garantia de adesão.

A conclusão, registrada com candura no `USO_DE_IA.md` e no relatório de Davi, é que o **requisito 4.3 está atendido**: function calling foi implementado, cabeado, com capacidade comprovada por diagnóstico isolado; a não-adesão sob contexto RAG denso é uma característica do modelo e da plataforma, não da implementação; o fallback estrutural garante citações corretas em qualquer caminho.

8.2 Limitações de modelagem

- **Heurística chars/4 para contagem de tokens** subestima sistematicamente PT-BR tokenizado por WordPiece. O backstop real não é o chunker (que mede com a mesma heurística defeituosa) mas a fronteira de erro do chunk-worker, que captura HTTP-StatusError quando o Ollama recusa um chunk. Um tokenizer real (e.g., tiktoken ou o próprio tokenizer do `nomic-embed-text`) eliminaria a heurística, ao custo de uma dependência adicional. Pendente para um eventual B5+.
- **Cache L2 pós-rerank** paga rerank a cada hit (~8 s residual em CPU). Um L2 chaveado só por `sha(query)` pularia também o rerank, mas perderia a invariante “se o corpus muda, a chave muda” — um trade-off arquitetural deliberado (§4.5).
- **Idempotência por posição, não por conteúdo**. O `point_id` é derivado de `chunk_id = f"{doc_id}:{chunk_index}"`. Reprocessar o mesmo PDF gera o mesmo `point_id` (idempotente), mas re-particionar um documento ligeiramente diferente gera

point_ids diferentes mesmo se o conteúdo coincide com algum chunk antigo. Um id derivado de sha256(text) daria deduplicação por conteúdo, ao custo de não poder recuperar a ordem original do documento. Aceito como limitação.

8.3 Débitos técnicos conhecidos

- **_RedisLike duplicado** entre `src/shared/cache.py` e `src/shared/session.py`. Cada módulo define seu próprio Protocol mínimo; uma consolidação em `src/shared/` com cast no wiring eliminaria a duplicação (hoje `redis_client: Any` na fronteira).
- **Smoke não-idempotente no cache L2**. Rodar `make smoke` duas vezes seguidas mascara o teste do L2: a 1ª query da 2ª rodada já é hit da rodada anterior. Workaround manual: `redis-cli FLUSHALL` antes. Solução opcional: flush no início do smoke, com o trade-off de acoplar o teste ao Redis.
- **Modo 2 parcial**. PC2 (pc2-jm) confirmado via Tailscale direct; PC3 (abdon-workstation) pendente até a data deste documento. Os experimentos do B4 dependem dos três PCs ativos.

8.4 Escopo cortado deliberadamente

O **Exp 3 (Ollama vs vLLM, bônus +10)** foi o primeiro corte da lista de cortes pré-definida no `TOD0.md`, aplicado em 2026-05-24 por restrição de cronograma. O sistema continua íntegro sem ele; o que se perde é a comparação quantitativa entre o batching ingênuo do Ollama e o continuous batching do vLLM, que seria evidência adicional para a discussão de paralelismo no servidor de inferência.

9. Conclusão

O sistema cumpre os requisitos de 4.1 a 4.6 do enunciado e os entregáveis 8.1, 8.2, 8.4 e Extra. A distribuição é real: três PCs físicos, conectados por uma VPN privada (Tailscale), executando containers descritos em Terraform a partir de um inventário Ansible. O paralelismo é demonstrado em duas dimensões — dados (ingestão em duas filas com chunk-workers escaláveis) e tarefas (query-workers replicados consumindo de uma fila comum). A engenharia de contexto é explícita e versionada; a observabilidade é completa (métricas, logs, dashboards) e a tolerância a falhas atravessa quatro camadas (retry, DLQ, degraded mode, fallback estrutural).

O resultado mais valioso para a disciplina, mais do que os números do smoke ou a topologia funcionando, foi articular um **resultado negativo bem caracterizado** (a não-adesão do qwen2.5:7b ao function

calling sob contexto RAG denso) — um achado isolado por diagnóstico controlado, com hipótese articulada e alavancas remanescentes mapeadas para trabalho futuro. Engenharia de contexto, no fim, não é sobre fazer o modelo sempre se comportar; é sobre saber **quando e como** ele se comporta, e ter um caminho secundário pronto para quando não se comportar.

Apêndices

- **A. Prompts versionados** — todos os arquivos em prompts/, com frontmatter YAML (version, model_target, last_changed, notes). Versão corrente: v0.2.3-b2.
- **B. Declaração de uso de IA** — docs/USO_DE_IA.md. Inclui entrada datada de 2026-05-24 que registra a suspensão deliberada do §3.4 para esta entrega final.
- **C. Runbooks operacionais** — docs/setup-tailscale.md (mesh privada do trio), docs/setup-gpu-pcl.md (driver NVIDIA + Container Toolkit).
- **D. Evidências de execução** — data/b1-smoke.txt, data/b2-smoke.txt, data/b2-metrics.txt, data/b2-filas.png (screenshot das duas filas no RabbitMQ Management UI). Os gráficos dos experimentos B4 (data/exp1.png, etc.) ainda não foram coletados na data deste documento.
- **E. Relatórios individuais (entregável 8.4)** — docs/relatorios_aprendizagem/davi_cava, joao_miguel.md, pablo_abdon.md.